CIS 3370 · INTRUSION DETECTION SYSTEMS

Train a model in Kibana, then test it honestly

You will train a classifier on `KDDTrain+`, score `KDDTest+` with it, and read every number that says how well it did. The model in this guide reaches 99.8 percent accuracy on one file and 80.5 percent on the other. Your job is to explain the gap, not to admire the first number. Every figure here is a screenshot of the system you are about to run.

1. Train **2. Infer** **3. Measure** **4. Explain the gap**

0 Before you start

You need two indices and the data views that point at them.

INDEX	ROWS	WHAT IT IS
`nsl-kdd-train`	125,973	KDDTrain+, the file you learn from
`nsl-kdd-test`	22,544	KDDTest+, the exam. Do not train on it.

If you cannot see `is_attack`: that field is not in the raw NSL-KDD files. It is a label we add, true when `class` is anything other than `normal`. The CSV you were given now carries it as a 45th column. If your copy does not have it, or you loaded the data before the column existed, make it in Kibana instead and nothing else changes:

1. **Stack Management → Data Views**, open your data view.
2. **Add field**, name it `is_attack`, type **Boolean**.
3. Turn on **Set value** and paint this script:

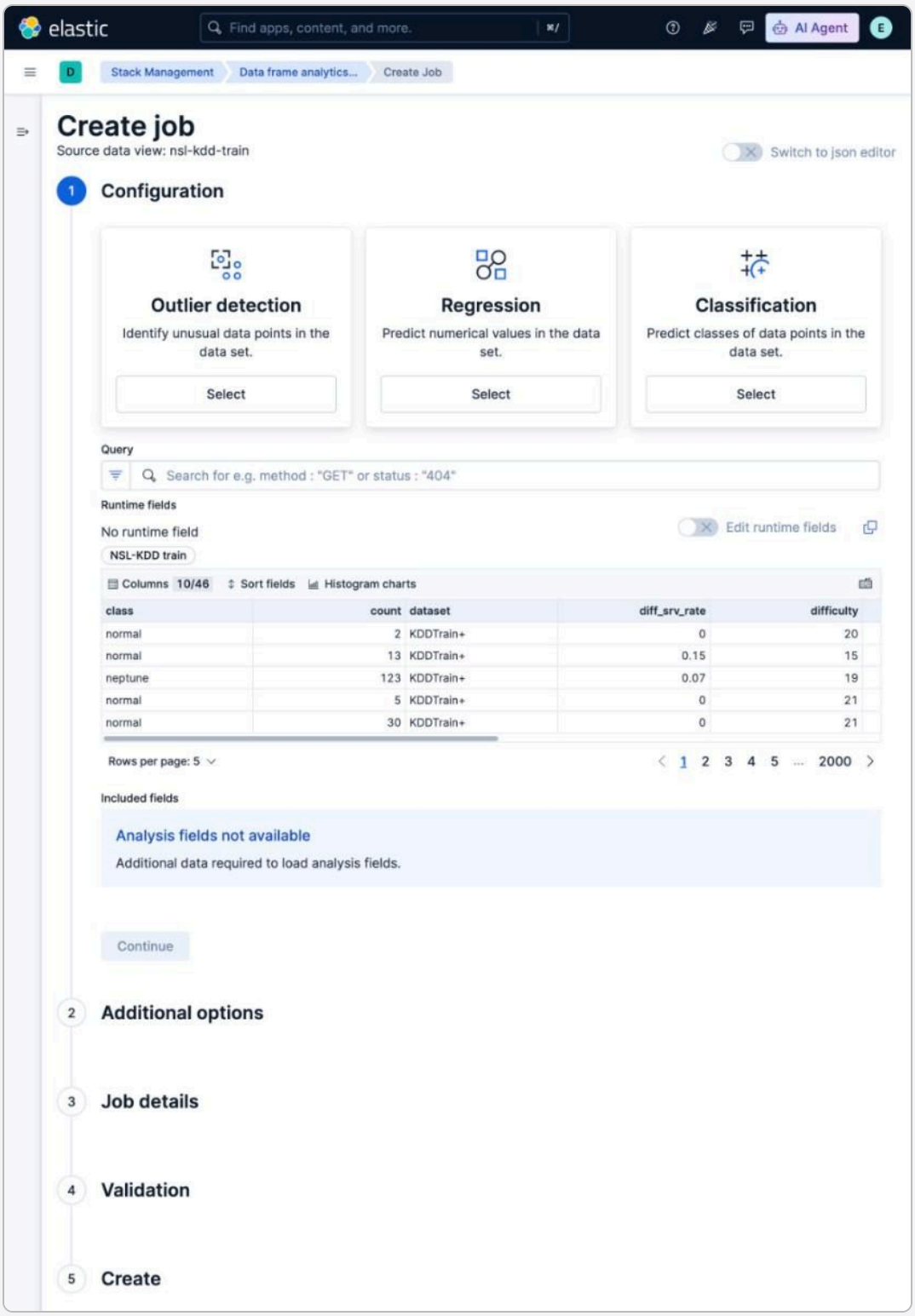
```
emit(doc['class'].value != 'normal');
```

4. Save. It now behaves like any other field in Discover, Lens and the dashboards.

A runtime field is computed when you query, so it costs nothing to store and can be changed at any time. The trade is speed on very large searches.

1 Create the training job

Kibana calls this a **data frame analytics** job. Open **Machine Learning → Data Frame Analytics → Create job** and pick the data view `nsl-kdd-train` as the source.



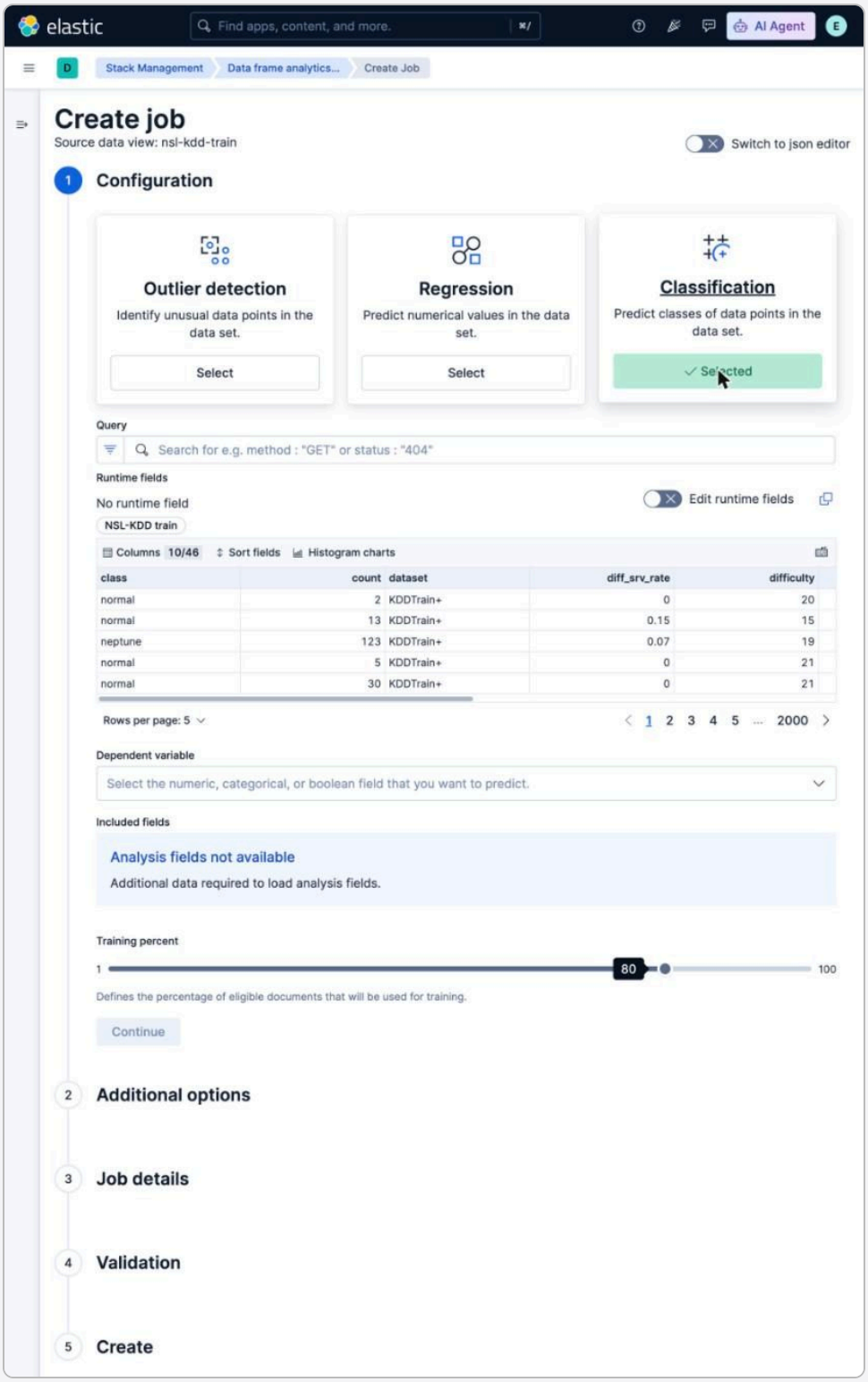
Create job, step 1. Three job types. Classification is the one with a label.

Three job types are offered, and the difference matters more than the names suggest.

TYPE	IT ANSWERS	NEEDS LABELS?
Outlier detection	which rows are unlike the rest	no
Regression	what number does this row have	yes, a number
Classification	which class does this row belong to	yes, a class

We have a label, so choose **Classification**.

> **Why not anomaly detection?** Kibana will refuse. Anomaly detection needs a time field and NSL-KDD has none, because the rows are connection records with no timestamp. Data frame analytics has no such requirement. That is the whole reason this job type exists for data like ours.



Classification selected. The dependent variable and the training percent slider appear below the data preview.

Fill the job in with exactly these settings.

SETTING	VALUE	WHY

Dependent variable	`is_attack`	the thing being predicted
Included fields	the 13 columns lecture 4 kept	everything else was measured and dropped
Training percent	20	train on a fifth, leave the rest to score
Destination index	`nsl-kdd-train-pred`	a copy of the rows with a prediction added
Job ID	`nsl-kdd-attack-classifier`	you will need it again for inference

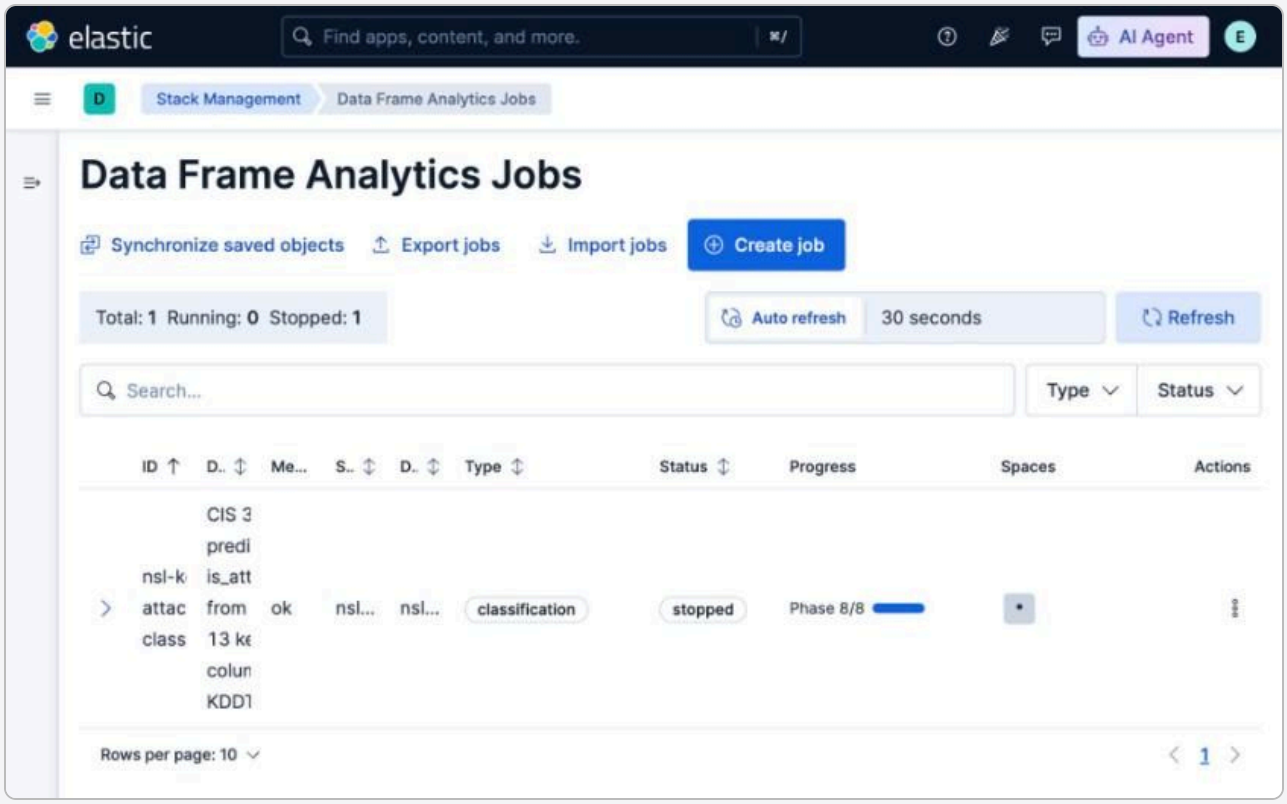
The wizard writes this request. You can run it directly in **Dev Tools** instead, and it is worth reading because the API is what the screens are made of.

```
PUT _ml/data_frame/analytics/nsl-kdd-attack-classifier
{
  "source": { "index": "nsl-kdd-train" },
  "dest":   { "index": "nsl-kdd-train-pred" },
  "analysis": {
    "classification": {
      "dependent_variable": "is_attack",
      "training_percent": 20,
      "randomize_seed": 42
    }
  },
  "analyzed_fields": { "includes": [
    "service","flag","logged_in","protocol_type","src_bytes","dst_bytes",
    "dst_host_srv_count","same_srv_rate","dst_host_diff_srv_rate","count",

"dst_host_serror_rate","dst_host_count","dst_host_srv_diff_host_rate","is_attack
  ] },
  "model_memory_limit": "1gb"
}

POST _ml/data_frame/analytics/nsl-kdd-attack-classifier/_start
```

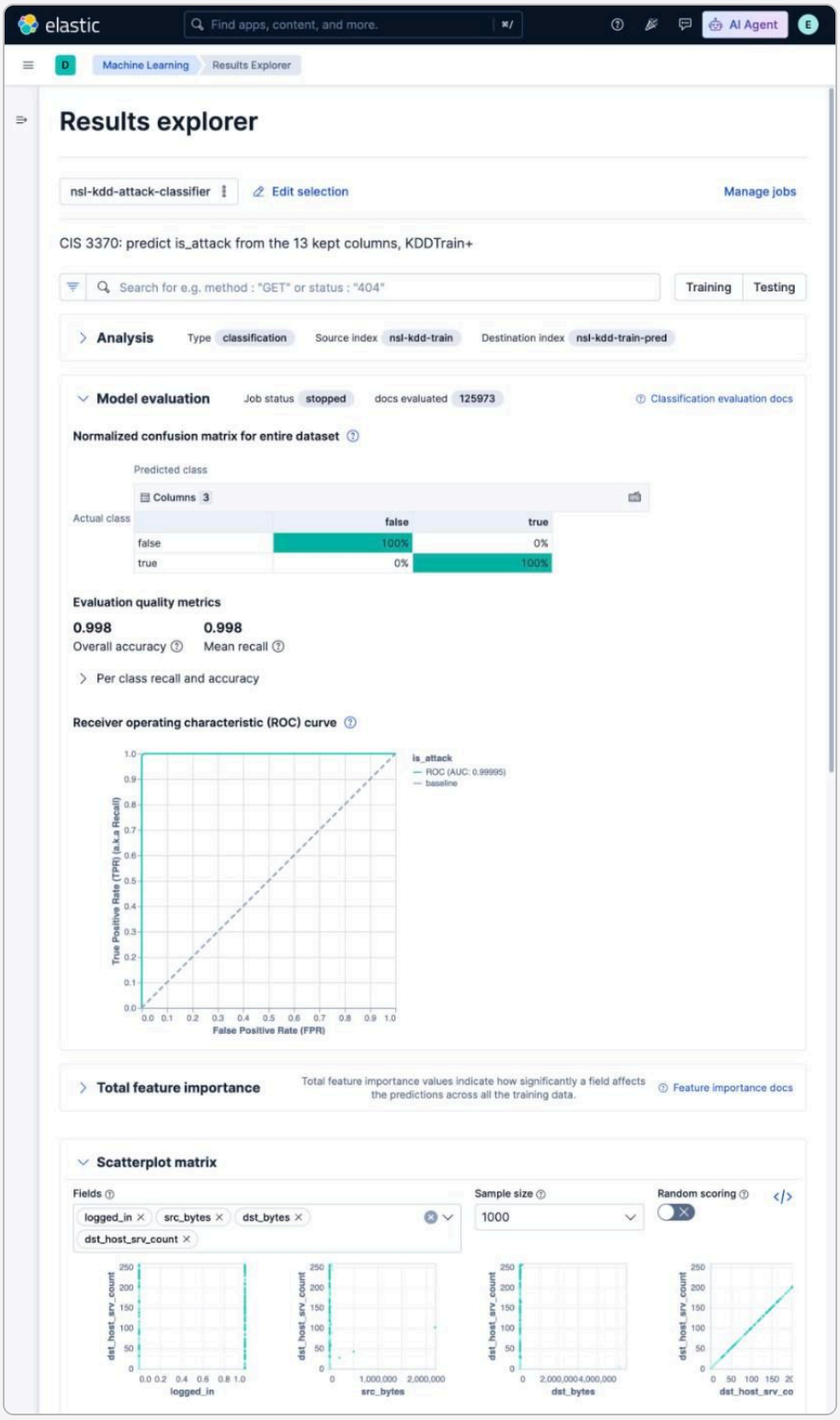
Starting the job runs eight phases in order: reindexing, loading data, feature selection, coarse parameter search, fine tuning parameters, final training, writing results, inference. When the list shows **Phase 8/8** and status `stopped`, it is finished.



The job list. Phase 8/8 and status stopped mean training and inference are both finished.

2 Read the results, and spot the trap in them

Open the job and choose **View results**.



Results explorer: normalized confusion matrix, quality metrics, the ROC curve, feature importance and a scatterplot matrix.

Four things are on that page.

- **Normalized confusion matrix.** Rows are what a connection really was, columns are what the model called it.
- **Evaluation quality metrics.** Overall accuracy and mean recall.

- **ROC curve and AUC.** AUC of 1.0 means a perfect ranking, 0.5 means a coin flip.
- **Total feature importance.** Which columns the model actually leaned on.

The trap. That matrix says 100 percent and 100 percent, and the heading says *for entire dataset*. The entire dataset includes the 20 percent the model was trained on, so the model is being graded on answers it memorised. Click the **Testing** toggle to exclude them. On the held-out 80 percent the honest numbers are accuracy 0.9976, precision 0.9988 and recall 0.9960. Still suspiciously high, and section 5 is about why.

3 Use the model on the file it has never seen

The job only scored rows inside its own source index. `KDDTest+` was never part of it, so the trained model has to be applied by hand. Three steps in **Dev Tools**.

Step 1. Find the model id the job produced. It is the job id with a timestamp on the end.

```
GET _ml/trained_models?size=50
```

Step 2. Put that model in an ingest pipeline. A pipeline is a set of steps every document passes through on its way into an index.

```
PUT _ingest/pipeline/nsl-kdd-attack-infer
{
  "processors": [
    { "inference": {
        "model_id": "nsl-kdd-attack-classifier-REPLACE_WITH_YOURS",
        "target_field": "ml",
        "inference_config": { "classification": { "num_top_classes": 2 } }
    } }
  ]
}
```

Step 3. Copy the test file through the pipeline into a new index. Create the destination with the same mapping as the source first, or Kibana will type `service` and `family` as text and every Terms aggregation on them will fail.

```
POST _reindex?wait_for_completion=true&refresh=true
{
  "source": { "index": "nsl-kdd-test" },
  "dest":   { "index": "nsl-kdd-test-pred", "pipeline": "nsl-kdd-attack-
```

```
infer" }
}
```

Every row in `nsl-kdd-test-pred` now carries `ml.is_attack_prediction` and `ml.prediction_probability` beside its real `is_attack`. Score it with the same evaluation API the results page uses:

```
POST _ml/data_frame/_evaluate
{
  "index": "nsl-kdd-test-pred",
  "evaluation": { "classification": {
      "actual_field": "is_attack",
      "predicted_field": "ml.is_attack_prediction",
      "metrics": { "multiclass_confusion_matrix": {}, "accuracy": {},
                   "precision": {}, "recall": {} }
  } }
}
```

4 The four boxes, and every rate that comes out of them

Every number below is a ratio of two of four counts. Learn the four counts and you never have to memorise a formula again.

COUNT	MEANING	HELD-OUT KDDTRAIN+	KDDTEST+
TP	attack, called attack	46,715	8,719
FN	attack, called normal: the miss	189	4,114
FP	normal, called attack: the false alarm	55	286
TN	normal, called normal	53,820	9,425

METRIC	FORMULA	HELD-OUT KDDTRAIN+	KDDTEST+
Precision	TP / (TP + FP)	0.9988	0.9682
Recall, TPR, sensitivity	TP / (TP + FN)	0.9960	0.6794
F1	2PR / (P + R)	0.9974	0.7985
Specificity, TNR	TN / (TN + FP)	0.9990	0.9705
FPR	FP / (FP + TN)	0.0010	0.0295

FNR	FN / (FN + TP)	0.0040	0.3206
Accuracy	(TP + TN) / all	0.9976	0.8048
Balanced accuracy	(TPR + TNR) / 2	0.9975	0.8250
MCC	uses all four boxes	0.9951	0.6571

Accuracy fell by 19 points. Recall fell by 32. When a dataset is unbalanced, accuracy is the number that hides the most, which is why balanced accuracy and MCC are on this table.

Caught and missed, per attack family, on the test file:

FAMILY	ATTACK ROWS	CAUGHT	RECALL
DoS	7,460	6,344	0.850
Probe	2,421	1,916	0.791
R2L	2,885	458	0.159
U2R	67	1	0.015

5 Why does it do so badly? Answer these

Write one paragraph for each, with the query or screenshot that supports it.

QUESTION 1

Accuracy on held-out rows of the training file is 0.9976. Those rows were never trained on. Why is that still not evidence that the model works?

QUESTION 2

In KDDTrain+, 73 percent of rows share their protocol, service, flag and both byte counts with at least one other row, and one single combination covers 12,795 rows. What does a random 20 and 80 split do when the same connection appears on both sides of it? What would a split that respects those groups look like?

QUESTION 3

Recall falls from 0.996 to 0.679. Using the family table, say which families carry the loss. Then open the dashboard panel that shows caught and missed by family and explain why those families and not the others.

QUESTION 4

R2L is 0.8 percent of the training file and 12.8 percent of the test file. Is this model bad, or is it being asked a different question than the one it was trained on? Defend your answer with numbers.

QUESTION 5

Precision stays high, 0.968, while recall collapses. In plain words, what kind of mistake is the model making, and what kind is it avoiding? Which of the two would you rather have in a real detection system, and why does the answer depend on who reads the alerts?

QUESTION 6

The test file contains 38 distinct attack names and the training file 23. The extra ones were never seen. Is detecting an attack type that did not exist at training time a classification problem at all? If not, what kind of problem is it, and which Elastic job type matches it?

QUESTION 7

FPR on the test file is 0.0295. A medium network sees roughly 100,000 benign connections a day. How many false alarms is that? One analyst can triage about 50 alerts in a shift. What has to change before this model can be deployed?

QUESTION 8

The top services behind the false negatives are telnet carrying processtable, guess_passwd and mscan. What do those three attacks have in common at the connection level, and which of the 13 features should have caught them?

6 Now change something and prove it

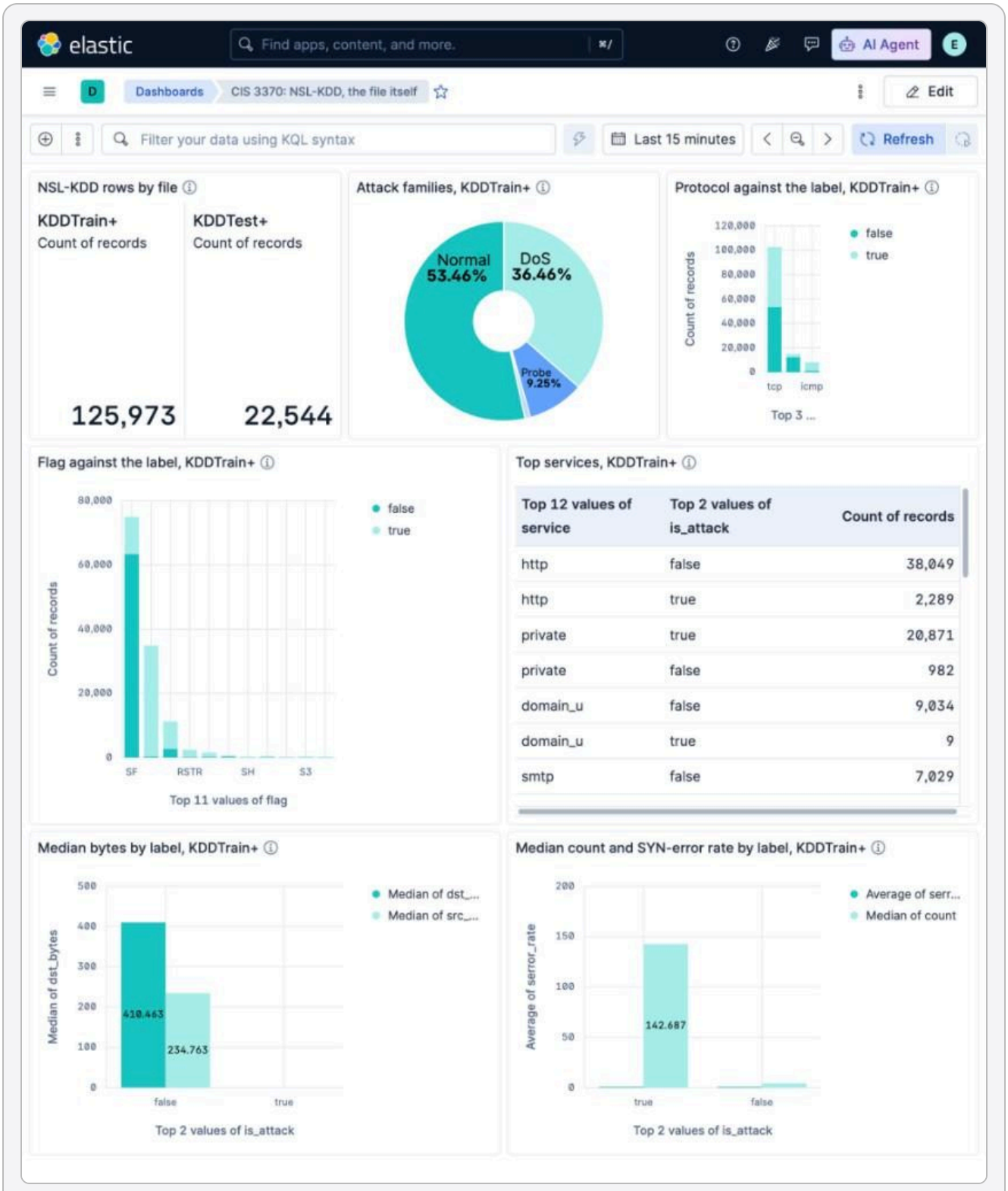
Pick two. Rerun the job, score the test file again, and report the four boxes before and after. A result that makes things worse still counts, as long as you can say why.

1. **Leak on purpose.** Add `difficulty` to the included fields and retrain. Report the new accuracy and explain, in one paragraph, why that number is worthless.
2. **Train on more.** Move training percent from 20 to 80. Does the test score improve? What does your answer say about whether the problem was ever the amount of data?
3. **Balance the classes.** R2L and U2R are rare in training. Find a way to give them more weight, or train only on the families the test file is full of, and measure what it costs the others.

4. **Move the line.** The model emits `ml.prediction_probability`. Instead of accepting its yes or no, pick your own cut: call anything above 0.3 an attack. Rebuild the four boxes at that cut and at 0.7. Plot recall against false alarms per day and choose a point you would defend.

7 The dashboards, and how to build a panel yourself

Four dashboards ship with this lab. Open each one, then rebuild at least one panel from scratch so you know what the pictures are made of.



Dashboard: NSL-KDD, the file itself.

What to look for: the two row counts, the family mix, and the three nominal columns that carry the label. The table of services is the same contingency table the chi-square walkthrough used, only wider.



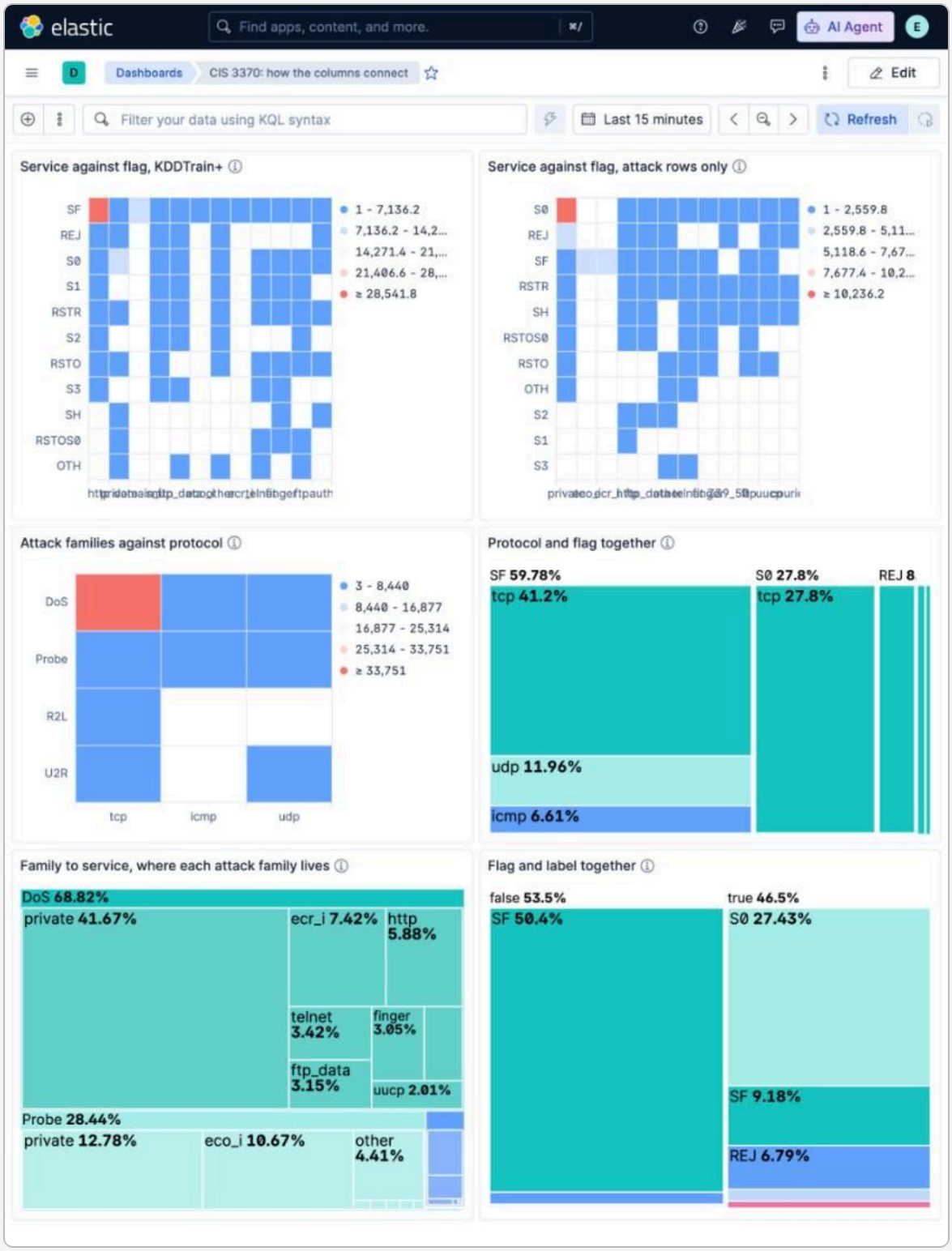
Dashboard: NSL-KDD, train against test.

What to look for: this is the slide that explains section 5. The family mix moves between the two files, 23 attack names become 38, and the two flood signals halve.



Dashboard: how good is the model.

What to look for: the four boxes as a table, recall and specificity as two bars, and the per-family split that tells you which attacks walked through.

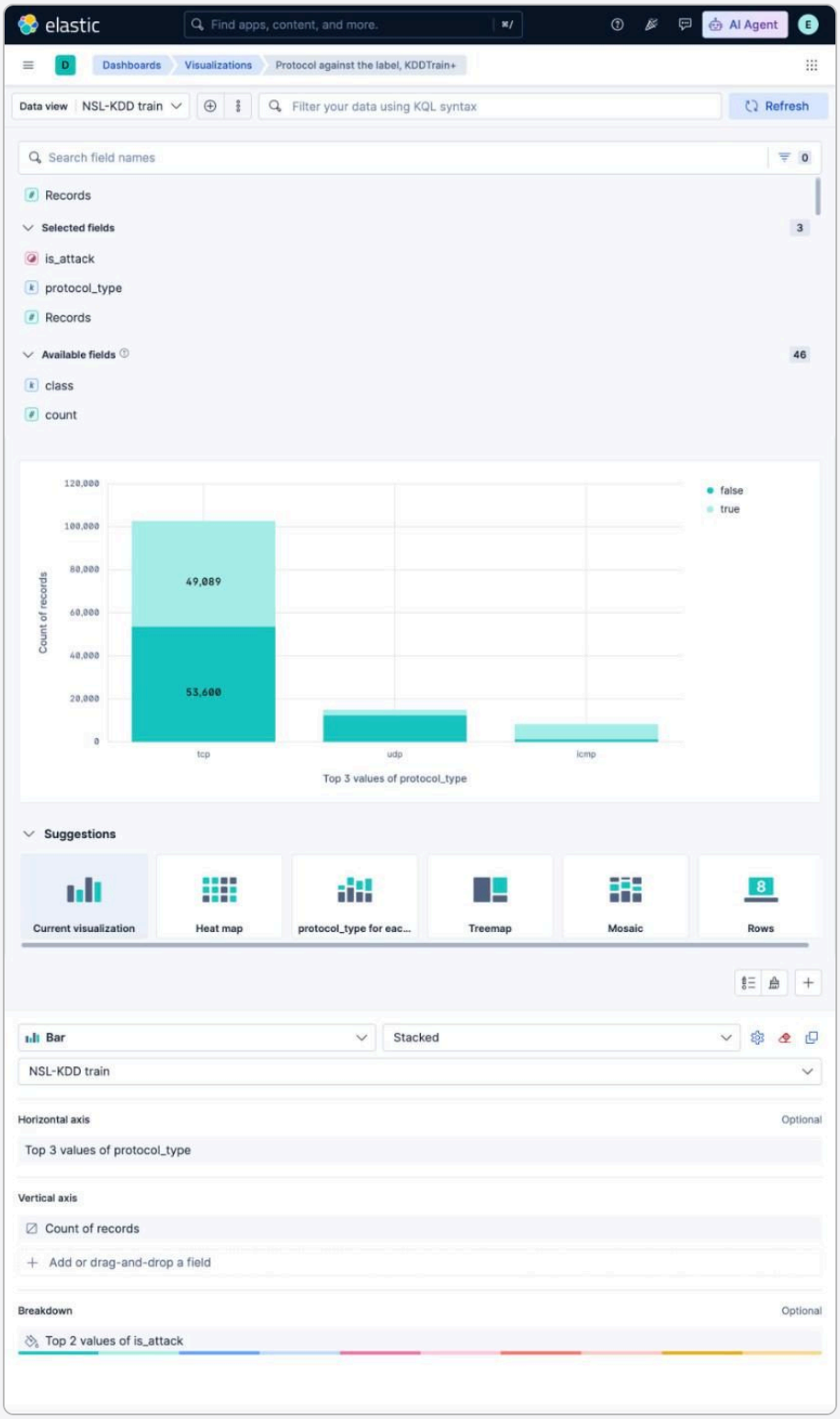


Dashboard: how the columns connect.

What to look for: which values occur together. The heat maps show service against flag for all rows and then for attack rows only. The difference between those two pictures is the attack signature.

Building a panel

Every Lens panel is four choices, and the editor is laid out in that order.



The Lens editor. Fields on the left, the chart in the middle, the four choices at the bottom.

CHOICE	WHERE	IN THIS PANEL
Which rows	data view and the KQL bar	`NSL-KDD train`, no filter

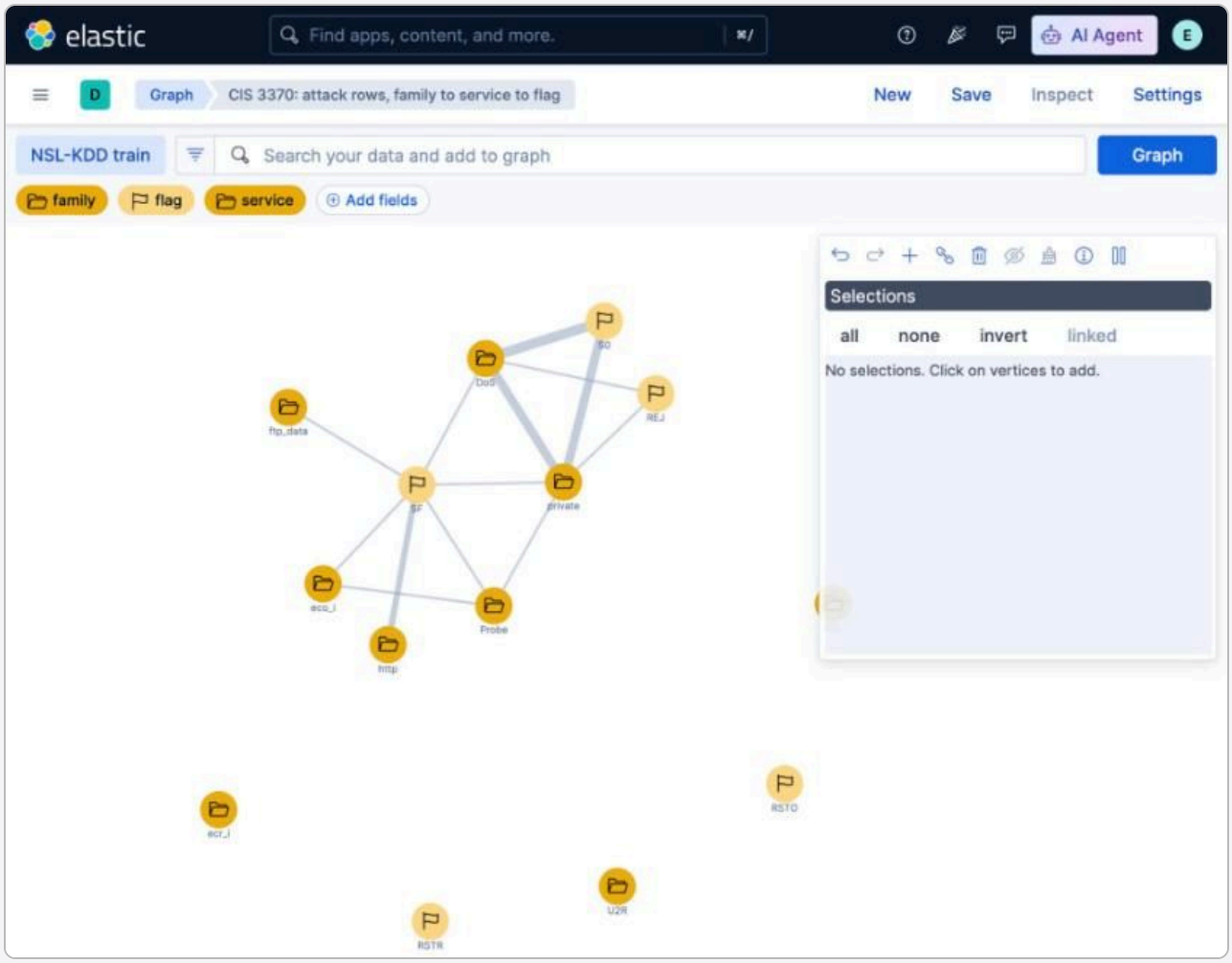
How to group them	Horizontal axis	Top 3 values of `protocol_type`
What to measure	Vertical axis	Count of records
How to split them	Breakdown	Top 2 values of `is_attack`

The **Suggestions** strip under the chart offers the same four choices drawn as a heat map, a treemap, a mosaic or a table. Click through them: the question has not changed, only the shape of the answer. Choose the shape that makes the comparison you care about easiest to see, then press **Save and return**.

A panel that needs you to explain it out loud is not finished. Give it a title that states the question, and a number on each bar.

8 The graph: which values travel together

The **Graph** app draws values as nodes and co-occurrence as edges. It only works on keyword fields, so `protocol_type`, `service`, `flag`, `class` and `family` are available and every rate and count is not.



The Graph app on attack rows, with family, service and flag as vertices.

Most people get an empty or scattered graph on the first try. Three settings and one button fix it.

SETTING	DEFAULT	USE	WHY
Significant links	on	**off**	on, it hides common pairs, which here is most of the data
Sample size	2,000	**10,000**	2,000 of 125,973 rows misses anything rare
Certainty	3	**1**	a value needs to appear in only one sampled document to count
Timeout	5,000 ms	**30,000 ms**	a bigger sample needs longer

Then the step nobody finds: run your query, click **all** in the Selections panel, and press **Add links between existing terms**, the chain icon. Graph places the nodes when you search but does not connect them until you ask. That single click turns a field of dots into the network above.

9 What to hand in

1. A screenshot of your finished job showing Phase 8/8, with your job id visible.
2. The four boxes for your model on the held-out training rows and on the test file, in one table.
3. Your answers to the eight questions in section 5, one paragraph each, each with its evidence.
4. The two experiments you chose from section 6, with before and after numbers.
5. One panel you built yourself, a screenshot of it, and one sentence saying what question it answers.
6. One sentence you would say to a manager who asks whether this model is ready to deploy.

Every number on this page was produced by running the job described here against `KDDTrain+` and `KDDTest+`. If your numbers differ, say by how much and why. A different random seed moves the last decimal, not the conclusion.